Compact U-Net for Solar Filament Instance Segmentation

A Solution to the Solar Filament Segmentation Challenge 2026

Microusada

ABSTRACT

This report describes our solution to the Solar Filament Segmentation Challenge 2026, a Kaggle competition on automatic segmentation of solar filaments in GONG H-α observations. We train a 487,857-parameter U-Net from scratch on 558 observations using intensity, local contrast, and mask-only supervision. A threshold chosen on eight development observations is frozen before evaluating 141 held-out observations. Global panoptic quality is 0.318382 across 255 annotator-image comparisons; an exploratory observation-cluster bootstrap interval is [0.300871, 0.335262]. We show preselected holdout examples and export original-size instances for all 180 test images. Fragmentation, merges, and annotator disagreement remain limitations. These are local results, not leaderboard scores.

Evaluation status. All reported scores are measured locally; no Kaggle leaderboard score is asserted.

1 INTRODUCTION

This report goes over the details of our submitted solution to the Solar Filament Segmentation Challenge 2026 [3]. The challenge, launched on July 10, 2026, is organized by the Earth-Space AI Research (ESAIR) Lab[1]. This challenge utilizes the MAGFiLO dataset for evaluation of the segmentation algorithms [2].

Our image-to-instance pipeline uses only supplied images and training masks. Local-contrast and supervised pixel-forest references motivate a compact U-Net; separate development and holdout measurements distinguish model selection from evaluation.

2 METHODOLOGY

2.1 Data and separation of observations

The official input contains 707 training JPEG observations and 180 test images, each 2048×2048, unsigned 8-bit, and grayscale. The training JSON has 1,154 annotator-image records and 8,199 instance annotations. We rasterize its polygons with `pycocotools`, without approximate replacement adapters.

Before mask use, a seed-2026 hash-based filename split assigns 566 observations to training/development and 141 to holdout. All annotators of one filename stay together. Eight of the 566 are reserved for development, leaving 558 for training. Forest and prototype U-Net use 24 and 64 training observations, also excluding development. Test labels and non-mask annotation metadata are never features. Holdout evaluation follows model/threshold freezing, with no retuning.

[1] www.esairlab.com/

Kaggle Competition, 2026, kaggle.com/competitions/filament-segmentation-2026 2026.

2.2 Two reference methods

The contrast reference estimates background B by 61-pixel elliptical closing. Relative dark contrast is $\max(B-I, 0)/\max(B, 1)$; eight-connected hysteresis uses thresholds 0.14 and 0.084. It excludes a 32-pixel limb from the filled disk above intensity 12, retaining area ≥ 64 pixels, $\leq 2\%$ of disk area, and elongation ≥ 1.5.

The scikit-learn forest [5] uses 32 trees, maximum depth 12, minimum leaf size 5, two CPU jobs, and seed 2026. Features are intensity, Gaussian means/dark contrasts at scales 4/12, blackhat at sizes 15/41, standard deviations at sizes 9/31, and gradient magnitude. Its 98,304 sampled pixels target 35% positives, so scores are uncalibrated. Predictions use minimum area 300; an earlier area-64 attempt exceeded the instance cap and stopped.

2.3 Compact U-Net trained from scratch

Our implementation follows U-Net's encoder–decoder pattern [4]: 487,857 parameters, encoder widths $(16, 32, 64, 128)$, and three decoder stages. Blocks contain two bias-free 3×3 convolutions, four-group normalization, and SiLU. Downsampling uses 2×2 max pooling; decoding uses bilinear upsampling and concatenated skips. A 1×1 convolution gives one logit.

Images are area-resized to 512×512. Inputs are intensity divided by 255 and signed local contrast $(\mu - I)/\max(\mu, 0.05)$ clipped to $[-1, 1]$, using a 49-pixel Gaussian kernel with standard deviation 8. Targets union only a training observation's annotators, area-resample foreground coverage, and quantize to unsigned 8-bit before conversion to $[0, 1]$. Partial coverage preserves some thin-structure signal. Evaluation uses original separate annotator instances, never union targets.

Training uses AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 4, and seed 2026. The loss adds positive-weight-12 binary cross-entropy to soft Dice loss (smoothing 1). Paired random 90° rotations and horizontal flips augment images and targets. The cache is bounded at 256 MiB. No external images or pretrained weights are used. The private, Internet-off Kaggle session offers two Tesla T4s; the model uses only `cuda:0`.

The final run completes 24 epochs on 558 observations in 2,407.33 seconds (2,424.74 including development evaluation). We use final-epoch weights.

2.4 Original-size instance outputs

After sigmoid, U-Net probabilities are bilinearly upsampled to 2048 pixels. Probabilities are thresholded inside the largest filled disk above intensity 12, excluding a 16-pixel limb. Eight-connected components require area ≥ 300 pixels, $\leq 5\%$ of disk area, and elongation ≥ 1.2. More than 256 components causes failure instead of silent truncation. Of predefined thresholds 0.45, 0.65, and 0.80, development PQ selects 0.65. We hash and freeze weights, configuration, sources, split, and threshold before holdout evaluation and test inference. No later retuning occurs.

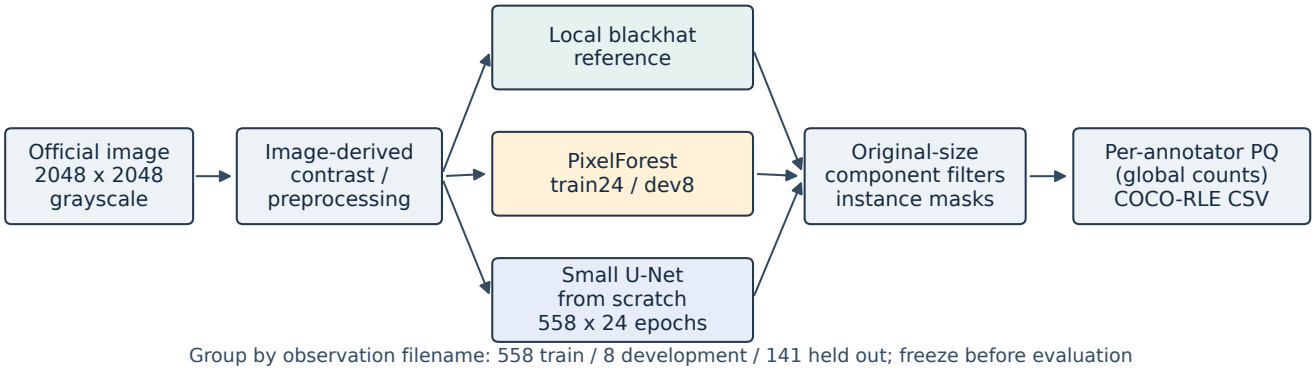


Figure 1: Image-only inference branches and shared original-size instance evaluation. Annotation unions supervise training only; separate annotators remain separate evaluation units. The U-Net is the final model, while contrast and forest branches are development references.

The CSV has `filament_id` (image stem plus instance suffix) and `segmentation_rle` (compressed COCO counts at fixed 2048×2048 size). Polygon input and compressed-RLE output are distinct formats. Empty predictions generate no invented filaments.

3 EVALUATION

3.1 Metric and evaluation units

The organizer's self-evaluation notebook v6 defines hits by strict $\mathrm{IoU}(p, g) > 0.5$ [1]. We retain its hit-pair behavior without adding Hungarian assignment. Counts and hit IoUs are pooled across annotator-image records:

$$\mathrm{PQ} = \frac{\sum_{(p,g)\ \mathrm{hit}} \mathrm{IoU}(p, g)}{\mathrm{TP} + \frac{1}{2}\mathrm{FP} + \frac{1}{2}\mathrm{FN}}. \tag{1}$$

A zero denominator returns zero. Every annotator's instances are compared separately with the same observation's predictions. This is global PQ, not an image-wise macro average. A bounded comparison against the organizer's visible functions on 30 tiny-mask/edge/random cases agrees on TP/FP/FN, with maximum case PQ difference 1.05×10^{-8}. It does not prove server equivalence. The actual holdout contains no excluded zero-GT sets.

Table 1: Exploratory development results on the same eight observations. RF and prototype U-Net use threshold 0.80; final U-Net uses 0.65. Training sizes differ: these are not controlled ablations or holdout results.

Method	TP	FP	FN	Global PQ
Contrast	2	762	112	0.002596
RF (24 train)	29	428	85	0.061846
U-Net (64 train)	23	256	91	0.070116
U-Net (558 train)	68	49	46	0.377220

3.2 Threshold selection and held-out result

On eight development observations, the final model gives PQ 0.331167, 0.377220, and 0.357211 at thresholds 0.45, 0.65, and 0.80 respectively. Selection of 0.65 precedes holdout access. Table 1 includes earlier references, but repeated diagnosis on this small subset and different training sizes prevent claims of significant superiority.

The frozen model gives **global PQ 0.3183818642** on 141 held-out observations and 255 annotator-image comparisons: TP = 952, FP = 1,200, FN = 712, and summed hit IoU = 607.4725969. A 1,000-resample percentile bootstrap (seed 2026) keeps all annotators of each observation together and recomputes pooled PQ. Its exploratory 95% interval is $[0.300871, 0.335262]$. Inference is not rerun for resampling. Hash-based splitting does not establish temporal independence, and correlated observations may invalidate nominal coverage; this is not a guaranteed confidence bound or leaderboard estimate.

3.3 Qualitative behavior and limitations

Figure 2 shows one useful detection and two difficult examples, with misses and excess components. Downsampling loses thin detail; union supervision and connected components cannot reliably separate touching/crossing instances. Annotator disagreement affects scores. Compactness and mask-only supervision do not establish instance separation or temporal robustness. No new model is selected after these outputs.

3.4 Reproducibility and verified export

Native software is Python 3.13.16, PyTorch 2.11.0+cu130, NumPy 2.1.3, SciPy 1.16.3, OpenCV 4.14.0, and `pycocotools` 2.0.11. Delivery manifests record exact hashes, weights, versions, splits, training, threshold, metrics, and examples, excluding raw data.

All 180 test images produce 1,661 unique instance rows; two zero-detection images are recorded. The 390,234-byte CSV re-reads successfully, and all 1,661 compressed-RLE masks round-trip with native COCO decoding at original size. Local export checks do not establish server scoring equivalence.

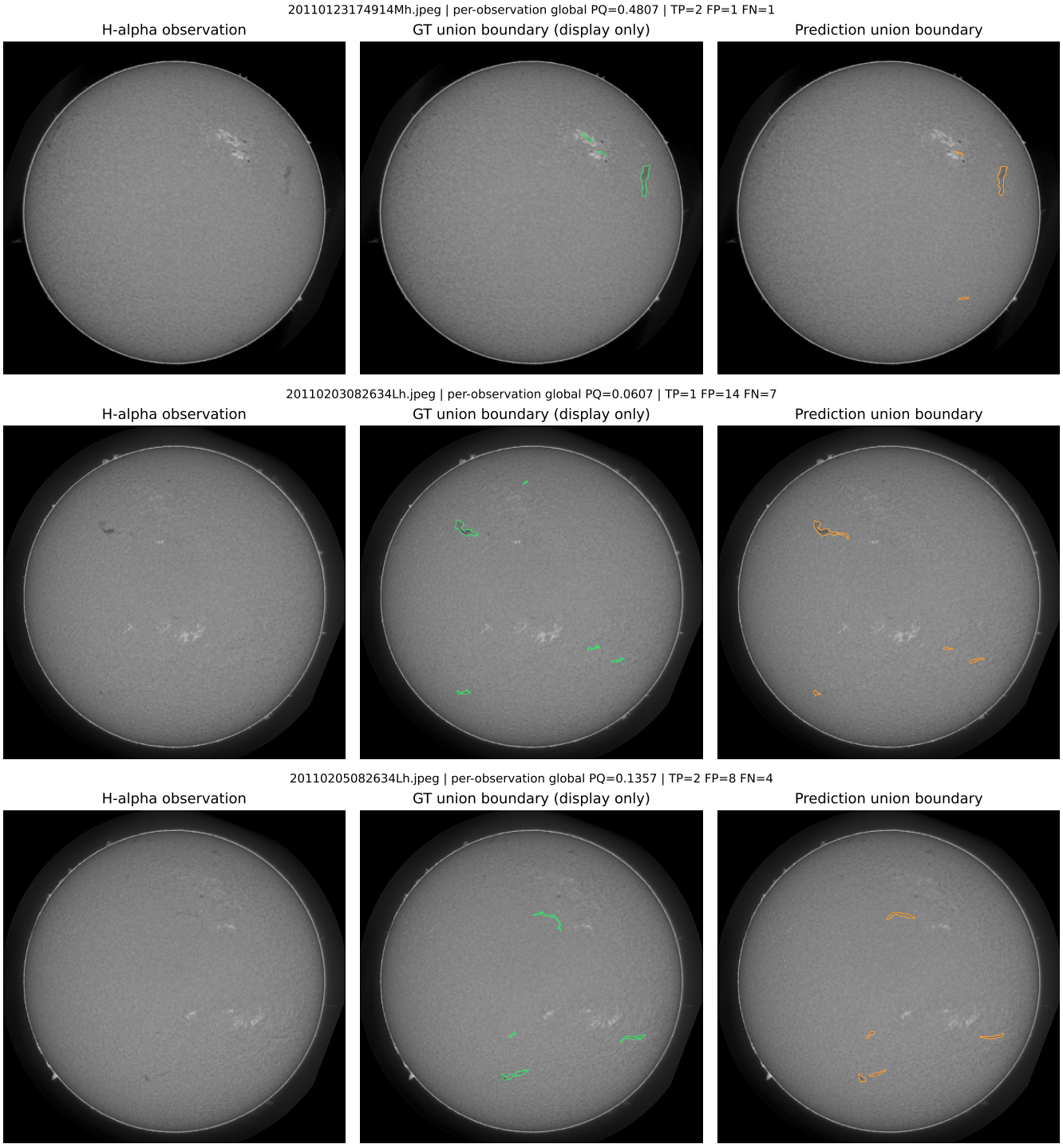


Figure 2: Actual frozen-model outputs for the first three sorted holdout filenames, fixed before evaluation. Each row shows the observation, green ground-truth union boundary (display only), and orange prediction union boundary. Separate annotators, not unions, are scored. Per-observation PQ is 0.4807, 0.0607, and 0.1357 from top to bottom. The lower-scoring examples expose false detections and incomplete or mismatched filament extent; these are not selected high-score cases. Data/annotations: MAGFiLO / NSO GONG H-α, CC BY-NC 4.0; noncommercial competition report.

4 ACKNOWLEDGMENT

The organization of the Filament Segmentation Challenge 2026 is partially supported by the U.S. National Science Foundation (NSF) under Grant No. 2209912 and 2433781, directorate for Computer and Information Science and Engineering (CSE), and Office of Advanced Cyberinfrastructure (OAC), and Grant No. 2511630, AST Division Of Astronomical Sciences and MPS Directorate for Mathematical and Physical Sciences.

The Filament Segmentation Challenge 2026 is sponsored by the U.S. National Science Foundation (NSF) National Solar Observatory (NSO).

This work utilizes GONG data obtained by the NSO Integrated Synoptic Program, managed by the National Solar Observatory, which is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the National Science Foundation and with contribution from the National Oceanic and Atmospheric Administration. The GONG network of instruments is hosted by the Big Bear Solar Observatory, High Altitude Observatory, Learmonth Solar Observatory, Udaipur Solar Observatory, Instituto de Astrofísica de Canarias, and Cerro Tololo Interamerican Observatory.

REFERENCES

[1] Azim Ahmadzadeh. 2026. Self-Evaluation Notebook. https://www.kaggle.com/code/azimahmadzadeh/self-evaluation-notebook. Organizer notebook, version 6; read 2026-11-03.

[2] Azim Ahmadzadeh, Rohan Adhyapak, Kartik Chaurasiya, Laxmi Alekhya Nagubandi, V. Aparna, Petrus C. Martens, Alexei Pevtsov, Luca Bertello, Alexander Pevtsov, Naomi Douglas, Samuel McDonald, Apaar Bawa, Eugene Kang, Riley Wu, Dustin J. Kempton, Aya Abdelkarem, Patrick M. Copeland, and Sri Harsha Seelamneni. 2024. A dataset of manually annotated filaments from H-alpha observations. In *Nature Scientific Data (Nature Scientific Data, Vol. 11)*. Springer, Article 1031, 1031 pages. https://doi.org/10.1038/s41597-024-03876-y

[3] Azim Ahmadzadeh, Dustin J. Kempton, Qin Li, and Alexei A. Pevtsov. 2026. Solar Filament Segmentation Challenge 2026. https://kaggle.com/competitions/filament-segmentation-2026. Kaggle.

[4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. https://arxiv.org/abs/1505.04597. arXiv:1505.04597; original architecture reference, not external pretrained weights.

[5] scikit-learn developers. 2026. RandomForestClassifier API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html. Accessed 2026-11-03; API reference, not a claim about the installed version.